PROJET D'ÉQUIPE · ANNÉE UNIVERSITAIRE 2026 – 2027

MediCare

Plateforme intelligente de gestion d'une clinique médicale

Journal et catalogue des prompts IA
Prompts envoyés · Prompts par module · Fiches de suivi

Assistant IA : Claude (Anthropic) · Claude Code

ÉQUIPE ET RÉPARTITION DES MODULES

MOD.	MODULE	PROMPTS	RESPONSABLE
M1	Utilisateurs, Patients & Médecins	M1-P1 → P4	
M2	Rendez-vous, Planning & Notifications	M2-P1 → P4	
M3	Consultations, Ordonnances & Facturation	M3-P1 → P4	
M4	Examens & Laboratoire	M4-P1 → P4	
M5	IA & Aide à la décision	M5-P1 → P4	
M6	Pharmacie & Stock	M6-P1 → P4	

ENCADRÉ PAR
_____

ÉTABLISSEMENT
_____

DATE
Octobre 2026

Sommaire

Introduction 3

Chapitre 1 — Méthode de rédaction des prompts 4

Chapitre 2 — Journal des prompts envoyés 5

2.1 Prompt 1 — Découpage du projet en six modules 5

2.2 Prompt 2 — Rapport d'analyse, de conception et de planification 5

2.3 Prompt 3 — Présentation des prompts par module 6

Chapitre 3 — Module M1 — Utilisateurs, Patients & Médecins 7

3.1 Présentation du module 7

3.2 Prompt M1-P1 — Analyse et conception 7

3.3 Prompt M1-P2 — Développement backend 8

3.4 Prompt M1-P3 — Développement frontend 9

3.5 Prompt M1-P4 — Tests et démonstration 10

3.6 Fiche de suivi des prompts 11

Chapitre 4 — Module M2 — Rendez-vous, Planning & Notifications 12

4.1 Présentation du module 12

4.2 Prompt M2-P1 — Analyse et conception 12

4.3 Prompt M2-P2 — Développement backend 13

4.4 Prompt M2-P3 — Développement frontend 15

4.5 Prompt M2-P4 — Tests et démonstration 16

4.6 Fiche de suivi des prompts 16

Chapitre 5 — Module M3 — Consultations, Ordonnances & Facturation 17

5.1 Présentation du module 17

5.2 Prompt M3-P1 — Analyse et conception 17

5.3 Prompt M3-P2 — Développement backend 18

5.4 Prompt M3-P3 — Développement frontend 19

5.5 Prompt M3-P4 — Tests et démonstration 20

5.6 Fiche de suivi des prompts 21

Chapitre 6 — Module M4 — Examens & Laboratoire 22

6.1 Présentation du module 22

6.2 Prompt M4-P1 — Analyse et conception 22

6.3 Prompt M4-P2 — Développement backend 23

6.4 Prompt M4-P3 — Développement frontend 24

6.5 Prompt M4-P4 — Tests et démonstration 25

6.6 Fiche de suivi des prompts 26

Chapitre 7 — Module M5 — IA & Aide à la décision 27

7.1 Présentation du module 27

7.2 Prompt M5-P1 — Analyse et conception 27

7.3 Prompt M5-P2 — Développement backend 28

7.4 Prompt M5-P3 — Développement frontend 29
7.5 Prompt M5-P4 — Tests et démonstration 30
7.6 Fiche de suivi des prompts 31

Chapitre 8 — Module M6 — Pharmacie & Stock 32
8.1 Présentation du module 32
8.2 Prompt M6-P1 — Analyse et conception 32
8.3 Prompt M6-P2 — Développement backend 33
8.4 Prompt M6-P3 — Développement frontend 34
8.5 Prompt M6-P4 — Tests et démonstration 35
8.6 Fiche de suivi des prompts 36

Introduction

Ce document rassemble les **prompts** (instructions données à l'assistant d'intelligence artificielle Claude, d'Anthropic, utilisé via Claude Code) dans le cadre du projet MediCare. Il a deux objectifs : rendre l'usage de l'IA **transparent** pour l'enseignant, et fournir à chacun des six membres de l'équipe les prompts de **son module**.

> **Organisation du document.** Le chapitre 2 reproduit **à l'identique** les prompts réellement envoyés pendant la phase d'analyse et de conception, avec le résultat obtenu. Les chapitres 3 à 8 présentent, pour chaque module, quatre prompts prêts à l'emploi (conception, backend, frontend, tests) que le responsable du module utilise pour son sprint, ainsi qu'une fiche de suivi où il note la date et le résultat de chaque utilisation.

Module	Intitulé	Sprint	Responsable	Prompts
M1	Utilisateurs, Patients & Médecins	Sprint 1	_____	M1-P1 à M1-P4
M2	Rendez-vous, Planning & Notifications	Sprint 2	_____	M2-P1 à M2-P4
M3	Consultations, Ordonnances & Facturation	Sprint 3	_____	M3-P1 à M3-P4
M4	Examens & Laboratoire	Sprint 4	_____	M4-P1 à M4-P4
M5	IA & Aide à la décision	Sprint 5	_____	M5-P1 à M5-P4
M6	Pharmacie & Stock	Sprint 6	_____	M6-P1 à M6-P4

Tableau 1 — Répartition des prompts par module

CHAPITRE 1

Méthode de rédaction des prompts

Chaque prompt suit la même structure, afin d'obtenir des réponses précises, cohérentes entre les modules et vérifiables :

Élément	Rôle dans le prompt	Exemple
Rôle	Fixe l'expertise attendue de l'assistant.	« Tu es développeur backend Java senior. »
Contexte	Décrit le projet, la pile technique et l'existant.	Spring Boot 3.2, Angular 17, MySQL 8, JWT, rôles.
Tâche	Liste numérotée de ce qu'il faut produire.	Entités, services, contrôleurs, DTO…
Contraintes	Limites de périmètre, règles métier, sécurité.	Ne pas modifier les autres modules, aucun secret.
Format de sortie	Forme exacte du résultat attendu.	PlantUML, tableau, code complet avec chemins.

Tableau 1.1 — Structure d'un prompt

Bonnes pratiques adoptées par l'équipe

- Un prompt = un module et une phase : on évite les demandes trop larges.
- Le contexte technique est répété dans chaque prompt pour qu'il soit utilisable seul.
- Les contrats entre modules (méthodes et endpoints partagés) sont rappelés explicitement.
- Le résultat est toujours relu, compilé et testé par le responsable avant d'être intégré : l'IA propose, l'équipe valide.
- Aucune donnée personnelle réelle ni aucun secret (mot de passe, clé d'API) n'est inclus dans un prompt.
- Chaque utilisation est notée dans la fiche de suivi du module (date, prompt, résultat, corrections apportées).

CHAPITRE 2

Journal des prompts envoyés

Les prompts ci-dessous sont reproduits **tels qu'ils ont été envoyés**, sans correction, dans l'ordre chronologique. Ils couvrent la phase d'analyse et de conception commune aux six modules.

2.1 Prompt 1 — Découpage du projet en six modules

DATE	12/10/2026
MODULES	Tous (M1 à M6)
LIVRABLE	`Plan de découpage (réponse dans la conversation)`

#1 Texte exact du prompt

> i want to ask u if we can add 2more modules to this project bro because we are 6 personnes and i want divise all the project to 6 module chacun a son module so give what is the plan

Résultat obtenu : Analyse du code existant (entités, contrôleurs, pages Angular) puis proposition d'un découpage en six modules : quatre modules existants regroupés (M1 à M4) et deux nouveaux modules (M5 IA & Aide à la décision, M6 Pharmacie & Stock), avec des règles de travail Git et un calendrier.

2.2 Prompt 2 — Rapport d'analyse, de conception et de planification

DATE	12/10/2026
MODULES	Tous (M1 à M6)
LIVRABLE	`docs/rapport/Rapport_MediCare_6_modules.pdf`

#2 Texte exact du prompt

> ok bro now i want u to give a rapport that contient this 6 modules Utilisateurs, Patients & Médecins existing Login/register, JWT, roles, SecurityConfig, users, patients, doctors, profile pages
> M2 Rendez-vous, Planning & Notifications existing Appointments (patient, secretary and doctor views), planning calendar, automatic reminders (RappelRdvService), the notification service and pages
> M3 Consultations, Ordonnances & Facturation existing Consultations, prescriptions, patient file and history, invoices and payments
> M4 Examens & Laboratoire existing Lab test requests, the lab area (/labo), entering results, the patient's lab results page
> M5 IA & Aide à la décision 🆕 new Takes over the current chatbot and the statistics/dashboard, then adds new AI features (below)
> M6 Pharmacie & Stock i want u to give les besoins fonctionnel et non fonctionnel diagramme de classe , use cas , backlog and then chaque module don't un sprint alone with there diagrams and there backlog let's do that all in pdf file and after we finish this i will tell u what we will do in the developpement

Résultat obtenu : Rapport PDF de 77 pages : besoins fonctionnels et non fonctionnels, diagramme de cas d'utilisation global, diagramme de classes global, product backlog (72 user stories), planification, puis un chapitre par sprint/module avec sprint

backlog, cas d'utilisation, description textuelle, diagramme de classes, diagramme de séquence, API REST et livrables.

2.3 Prompt 3 — Présentation des prompts par module

DATE	12/10/2026
MODULES	Tous (M1 à M6)
LIVRABLE	`docs/rapport/Prompts_MediCare_par_module.pdf`

#3 Texte exact du prompt

> ok bro now in this matiere we must affiche to the prof les prompts that give it to u so i want u to create for me a pdf file that conrient des prompt and divise that prompts with module so chaque module has his own prompt bro

Résultat obtenu : Le présent document : journal des prompts envoyés et catalogue de prompts par module (conception, backend, frontend, tests) avec une fiche de suivi pour chaque responsable.

CHAPITRE 3 · SPRINT 1

Module M1 — Utilisateurs, Patients & Médecins

3.1 Présentation du module

SPRINT	Sprint 1 — 3 semaines
STATUT	Existant
RESPONSABLE	_____
OBJECTIF	Disposer d'une gestion des identités fiable et sécurisée : inscription, authentification JWT, rôles (dont le nouveau rôle PHARMACIEN), comptes, médecins et fiches patients, avec traçabilité des actions sensibles.

User stories couvertes par les prompts

- US-1.1 : en tant que visiteur, je veux créer un compte patient afin de accéder à l'espace patient.
- US-1.2 : en tant qu'utilisateur, je veux m'authentifier par email et mot de passe afin de accéder à l'espace correspondant à mon rôle.
- US-1.3 : en tant qu'utilisateur, je veux me déconnecter afin de sécuriser ma session.
- US-1.4 : en tant qu'utilisateur, je veux consulter et modifier mon profil afin de garder mes informations à jour.
- US-1.5 : en tant qu'utilisateur, je veux changer mon mot de passe afin de sécuriser mon compte.
- US-1.6 : en tant qu'administrateur, je veux gérer les comptes utilisateurs (création, modification, suppression) afin de administrer les accès à la plateforme.
- US-1.7 : en tant qu'administrateur, je veux attribuer un rôle et activer ou désactiver un compte afin de contrôler les droits de chacun.
- US-1.8 : en tant qu'administrateur, je veux rattacher une secrétaire à un médecin afin de organiser le secrétariat par praticien.
- US-1.9 : en tant qu'administrateur, je veux gérer les médecins (spécialité, horaires, disponibilité) afin de tenir l'annuaire médical à jour.
- US-1.10 : en tant que secrétaire, je veux créer et modifier la fiche d'un patient afin de constituer son dossier administratif.
- US-1.11 : en tant que secrétaire ou médecin, je veux rechercher un patient par nom, n° de dossier ou téléphone afin de le retrouver rapidement.
- US-1.12 : en tant que patient, je veux consulter la liste des médecins et leurs spécialités afin de choisir mon praticien.
- US-1.13 : en tant qu'administrateur, je veux consulter le journal d'audit des actions sensibles afin de assurer la traçabilité.

3.2 Prompt M1-P1 — Analyse et conception

M1-P1 **Analyse et conception**

Conception — produit les besoins, les diagrammes UML et le sprint backlog du module.

> Tu es analyste-concepteur UML et Scrum Master.
> Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN.
>
> Je suis responsable du module M1 « Utilisateurs, Patients & Médecins », qui correspond au sprint 1 (3 semaines). Objectif du module : Disposer d'une gestion des identités fiable et sécurisée : inscription, authentification JWT, rôles (dont le nouveau rôle PHARMACIEN), comptes, médecins et fiches patients, avec traçabilité des actions sensibles.

Ce qui existe déjà dans le code :
- Inscription et connexion JWT (AuthController, JwtService)
- CRUD utilisateurs, médecins, patients
- Pages login, register, utilisateurs, médecins, patients, profil
- Guards Angular authGuard / roleGuard
Ce qu'il faut ajouter :
- Rôle PHARMACIEN et redirection /pharmacie
- Changement de mot de passe
- Recherche de patients
- Activation / désactivation des comptes
- Journal d'audit (JournalAudit)

Produis en français, pour ce module uniquement :
1. Les besoins fonctionnels (formulés « Le système doit… », identifiants BF-M1-01, BF-M1-02…) et les besoins non fonctionnels qui le concernent (sécurité, performance, ergonomie, fiabilité).
2. Le diagramme de cas d'utilisation en PlantUML. Acteurs : Visiteur, Utilisateur authentifié, Administrateur, Secrétaire, Médecin, Patient. Utilise les relations « include » et « extend » ; marque les cas nouveaux avec le stéréotype <<nouveau>>.
3. La description textuelle du cas « S'authentifier » : acteurs, pré-conditions, post-conditions, scénario nominal numéroté, scénarios alternatifs.
4. Le diagramme de classes en PlantUML (attributs typés Java, multiplicités, énumérations). Les classes d'autres modules apparaissent avec le stéréotype <<externe>> : aucune (M1 fournit les classes de référence Utilisateur, Patient et Medecin).
5. Le diagramme de séquence PlantUML du scénario « S'authentifier » à travers les couches Angular → Controller → Service → Repository.
6. Le sprint backlog sous forme de tableau : user stories « En tant que…, je veux… afin de… », priorité (Haute / Moyenne / Basse), story points (1, 2, 3, 5, 8) et tâches techniques estimées en heures.

Contraintes : reste dans le périmètre du module et respecte les contrats avec les autres modules :
- M1 fournit à Tous : Utilisateur, Role, Patient, Medecin ; sécurité JWT ; JournalAuditService.tracer(...)
- M1 utilise NotificationService.notifier(utilisateur, type, titre, message, referenceType, referenceId) (fourni par M2)

Résultat attendu : Besoins BF/BNF, diagrammes de cas d'utilisation, de classes et de séquence (PlantUML), description textuelle et sprint backlog.

3.3 Prompt M1-P2 — Développement backend

M1-P2 Développement backend

Développement — entités, services, contrôleurs REST et sécurité.

Tu es développeur backend Java senior.
Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN.

Je développe la partie backend du module M1 « Utilisateurs, Patients & Médecins ».
Fichiers concernés :

- entity : Utilisateur, Role, Patient, Medecin, JournalAudit (nouveau)
- controller : AuthController, UtilisateurController, PatientController, MedecinController
- service : AuthService, UtilisateurService, PatientService, MedecinService
- security : JwtService, JwtAuthenticationFilter, SecurityConfig, CustomUserDetailsService

Travail demandé :
1. Entités JPA : Utilisateur, Role, Patient, Medecin, JournalAudit (nouveau). Ajoute les nouveaux attributs et associations du diagramme de classes du module.
2. Repositories Spring Data avec les requêtes nécessaires.
3. Services métier, avec @Transactional dès que plusieurs entités sont modifiées. Règles métier à respecter :
- email unique et mot de passe haché avec BCrypt ;
- refuser la connexion d'un compte désactivé ;
- le changement de mot de passe exige l'ancien mot de passe ;
- numéro de dossier patient généré automatiquement ;
- ajouter la valeur PHARMACIEN à l'énumération Role et la redirection vers /pharmacie ;
- enregistrer une entrée JournalAudit à chaque création, modification ou suppression sensible.
4. Contrôleurs REST exposant exactement ces endpoints, sécurisés avec @PreAuthorize :
- POST /api/auth/register — Inscription d'un patient (Public)
- POST /api/auth/login — Authentification, renvoie le JWT et le rôle (Public)
- GET /api/utilisateurs — Liste des utilisateurs (ADMIN)
- GET / PUT /api/utilisateurs/{id} — Consulter / modifier un profil (Tous (soi-même))
- POST / DELETE /api/utilisateurs[/{id}] — Créer / supprimer un compte (ADMIN)
- PUT /api/utilisateurs/{id}/mot-de-passe — Changer son mot de passe (Tous (soi-même))
- PUT /api/utilisateurs/{id}/statut — Activer / désactiver un compte (ADMIN)
- GET / POST / PUT / DELETE /api/medecins[/{id}] — Gestion des médecins (ADMIN (lecture : tous))
- GET / POST / PUT / DELETE /api/patients[/{id}] — Gestion des fiches patients (ADMIN, SECRETAIRE, MEDECIN)
- GET /api/patients/recherche?q= — Recherche multicritère (ADMIN, SECRETAIRE, MEDECIN)
- GET /api/audit — Consultation du journal d'audit (ADMIN)
5. DTO d'entrée et de sortie (ne jamais renvoyer directement une entité) avec validation jakarta.validation.
6. Erreurs gérées par GlobalExceptionHandler : 400 (validation), 403, 404, 409 (conflit).

Contraintes : ne modifie pas les fichiers des autres modules (demande-moi si un contrat doit évoluer), utilise NotificationService.notifier(...) pour les notifications, aucun secret dans le code, endpoints visibles dans Swagger. Donne le code complet de chaque fichier avec son chemin.

Résultat attendu : Code Spring Boot complet du module, prêt à être relu, testé puis intégré par pull request.

3.4 Prompt M1-P3 — Développement frontend

M1-P3 **Développement frontend**

Développement — services Angular, écrans, routes et menus.

Tu es développeur frontend Angular 17 (composants standalone, formulaires réactifs, HttpClient, RxJS).
Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN. L'intercepteur authInterceptor ajoute le jeton JWT ; authGuard et roleGuard protègent les routes.

Je développe l'interface du module M1 « Utilisateurs, Patients & Médecins ».

Fichiers concernés :
- pages/login, pages/register, pages/home
- pages/back : utilisateurs, medecins, patients, profil, audit (nouveau)
- core : auth.service, auth.guard, role.guard, auth.interceptor, utilisateur.service (nouveau)

Travail demandé :
1. Crée core/utilisateur.service.ts avec une méthode typée par endpoint du module :
- POST /api/auth/register — Inscription d'un patient (Public)
- POST /api/auth/login — Authentification, renvoie le JWT et le rôle (Public)
- GET /api/utilisateurs — Liste des utilisateurs (ADMIN)
- GET / PUT /api/utilisateurs/{id} — Consulter / modifier un profil (Tous (soi-même))
- POST / DELETE /api/utilisateurs[/{id}] — Créer / supprimer un compte (ADMIN)
- PUT /api/utilisateurs/{id}/mot-de-passe — Changer son mot de passe (Tous (soi-même))
- PUT /api/utilisateurs/{id}/statut — Activer / désactiver un compte (ADMIN)
- GET / POST / PUT / DELETE /api/medecins[/{id}] — Gestion des médecins (ADMIN (lecture : tous))
- GET / POST / PUT / DELETE /api/patients[/{id}] — Gestion des fiches patients (ADMIN, SECRETAIRE, MEDECIN)
- GET /api/patients/recherche?q= — Recherche multicritère (ADMIN, SECRETAIRE, MEDECIN)
- GET /api/audit — Consultation du journal d'audit (ADMIN)
2. Crée ou mets à jour les écrans suivants :
- login et register : validations des champs et redirection selon le rôle (y compris /pharmacie) ;
- utilisateurs : liste paginée filtrable par rôle, création / édition, attribution du rôle, activation / désactivation ;
- medecins : liste, ajout, édition, disponibilité ;
- patients : formulaire complet (antécédents, allergies, groupe sanguin) et recherche avec temporisation (debounce) ;
- profil : édition des informations et boîte de dialogue « Changer le mot de passe » ;
- audit (nouveau) : journal filtrable par date, utilisateur et action.
3. Déclare les routes du module dans app.routes.ts (uniquement le bloc de ce module), protégées par roleGuard pour les rôles : ADMIN pour l'administration, tous les rôles pour le profil.
4. Ajoute les entrées de menu dans le ou les layouts concernés.

Exigences d'interface : en français, responsive (mobile, tablette, ordinateur), indicateurs de chargement, messages d'erreur clairs (403, 404, 409), confirmation avant suppression, badges de statut colorés, champs avec libellés (accessibilité). Donne le code complet (TypeScript, HTML, CSS) de chaque composant.

Résultat attendu : Composants Angular du module, service HTTP, routes protégées et entrées de menu.

3.5 Prompt M1-P4 — Tests et démonstration

M1-P4 **Tests et démonstration**

Validation — tests, données de démo et préparation de la sprint review.

Tu es ingénieur qualité logiciel.
Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN. Tests backend avec JUnit 5, Mockito et MockMvc.

Pour le module M1 « Utilisateurs, Patients & Médecins », écris :
1. Les tests unitaires des services du module (Mockito), cas nominaux et cas d'erreur, en particulier :

> - inscription avec un email déjà utilisé
> - connexion d'un compte désactivé
> - changement de mot de passe avec un ancien mot de passe erroné
> - recherche de patient par nom et par n° de dossier
> - création d'une entrée d'audit lors d'une suppression
> 2. Les tests d'intégration des endpoints (MockMvc) qui vérifient la sécurité par rôle : 401 sans jeton, 403 avec un rôle non autorisé, 200 / 201 avec le bon rôle.
> 3. Une méthode d'initialisation des données de démonstration du module à ajouter au DataSeeder, sans dupliquer les données existantes.
> 4. Le scénario pas à pas de la démonstration de fin de sprint : Démonstration : inscription → connexion → redirection par rôle pour les six rôles.
> 5. La checklist de la définition de « Terminé » appliquée à ce module (compilation, sécurité par rôle, Swagger, page accessible depuis le menu, données de démo, tests verts, revue de code, aucun secret).

Résultat attendu : Tests unitaires et d'intégration, données de démonstration, scénario de démo et checklist « Terminé ».

3.6 Fiche de suivi des prompts

À remplir par le responsable du module à chaque utilisation d'un prompt.

Date	Prompt	Résultat obtenu	Corrections apportées par l'équipe
	M1-P1		
	M1-P2		
	M1-P3		
	M1-P4		

Tableau 3.1 — Fiche de suivi — module M1

CHAPITRE 4 · SPRINT 2

Module M2 — Rendez-vous, Planning & Notifications

4.1 Présentation du module

SPRINT	Sprint 2 — 3 semaines
STATUT	Existant
RESPONSABLE	_____
OBJECTIF	Offrir une prise de rendez-vous fiable fondée sur les disponibilités réelles des médecins, sans double réservation, et un service de notifications central utilisé par tous les modules.

User stories couvertes par les prompts

- US-2.1 : en tant que patient, je veux prendre un rendez-vous en ligne en choisissant un médecin et un créneau libre afin de éviter les appels téléphoniques.
- US-2.2 : en tant que patient, je veux consulter mes rendez-vous à venir et passés afin de suivre mon agenda.
- US-2.3 : en tant que patient, je veux annuler ou reporter mon rendez-vous jusqu'à 24 h avant afin de libérer le créneau.
- US-2.4 : en tant que secrétaire, je veux créer, modifier et supprimer un rendez-vous pour un patient afin de gérer les demandes reçues à l'accueil.
- US-2.5 : en tant que secrétaire ou médecin, je veux confirmer un rendez-vous afin de informer le patient.
- US-2.6 : en tant que médecin, je veux consulter mon planning en vue jour, semaine ou mois afin de organiser ma journée.
- US-2.7 : en tant que médecin ou administrateur, je veux définir les plages horaires et les indisponibilités afin de n'ouvrir que les créneaux réels.
- US-2.8 : en tant que système, je veux empêcher la double réservation d'un créneau afin de garantir la cohérence du planning.
- US-2.9 : en tant que médecin, je veux marquer un patient absent afin de alimenter les statistiques d'absentéisme (M5).
- US-2.10 : en tant que système, je veux envoyer un rappel unique 24 h avant chaque rendez-vous afin de réduire les oublis sans saturer le patient.
- US-2.11 : en tant qu'utilisateur, je veux être notifié à la création, confirmation, annulation ou au report d'un rendez-vous afin de être informé en temps réel.
- US-2.12 : en tant qu'utilisateur, je veux consulter mes notifications, les marquer comme lues et les supprimer afin de gérer mes alertes.
- US-2.13 : en tant que patient, je veux recevoir aussi les rappels par email afin de ne pas manquer mon rendez-vous.

4.2 Prompt M2-P1 — Analyse et conception

M2-P1 Analyse et conception

Conception — produit les besoins, les diagrammes UML et le sprint backlog du module.

> Tu es analyste-concepteur UML et Scrum Master.
> Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN.
>
> Je suis responsable du module M2 « Rendez-vous, Planning & Notifications », qui correspond au sprint 2 (3 semaines).

Objectif du module : Offrir une prise de rendez-vous fiable fondée sur les disponibilités réelles des médecins, sans double réservation, et un service de notifications central utilisé par tous les modules.
Ce qui existe déjà dans le code :
- CRUD des rendez-vous (patient, secrétaire, médecin)
- Planning calendrier (PlanningsComponent)
- Rappels automatiques (RappelRdvService, cron horaire)
- Notifications : liste, non lues, compteur, lecture, suppression
Ce qu'il faut ajouter :
- Plages horaires et indisponibilités des médecins
- Calcul des créneaux libres et anti double réservation
- Annulation / report par le patient, statut ABSENT
- Rappel unique (correction : le rappel est actuellement renvoyé à chaque exécution horaire)
- Canal email pour les rappels

Produis en français, pour ce module uniquement :
1. Les besoins fonctionnels (formulés « Le système doit… », identifiants BF-M2-01, BF-M2-02…) et les besoins non fonctionnels qui le concernent (sécurité, performance, ergonomie, fiabilité).
2. Le diagramme de cas d'utilisation en PlantUML. Acteurs : Patient, Secrétaire, Médecin, Administrateur, Utilisateur authentifié, Système (planificateur). Utilise les relations « include » et « extend » ; marque les cas nouveaux avec le stéréotype <<nouveau>>.
3. La description textuelle du cas « Prendre un rendez-vous en ligne » : acteurs, pré-conditions, post-conditions, scénario nominal numéroté, scénarios alternatifs.
4. Le diagramme de classes en PlantUML (attributs typés Java, multiplicités, énumérations). Les classes d'autres modules apparaissent avec le stéréotype <<externe>> : Patient, Medecin, Utilisateur (module M1).
5. Le diagramme de séquence PlantUML du scénario « Prendre un rendez-vous en ligne » à travers les couches Angular → Controller → Service → Repository.
6. Le sprint backlog sous forme de tableau : user stories « En tant que…, je veux… afin de… », priorité (Haute / Moyenne / Basse), story points (1, 2, 3, 5, 8) et tâches techniques estimées en heures.

Contraintes : reste dans le périmètre du module et respecte les contrats avec les autres modules :
- M2 utilise Utilisateur, Role, Patient, Medecin ; sécurité JWT ; JournalAuditService.tracer(…) (fourni par M1)
- M2 fournit à Tous : NotificationService.notifier(utilisateur, type, titre, message, referenceType, referenceId)
- M2 fournit à M5 : Historique des RDV et statut ABSENT
- M2 utilise Médecin suggéré par le triage (fourni par M5)

Résultat attendu : Besoins BF/BNF, diagrammes de cas d'utilisation, de classes et de séquence (PlantUML), description textuelle et sprint backlog.

4.3 Prompt M2-P2 — Développement backend

M2-P2 **Développement backend**

Développement — entités, services, contrôleurs REST et sécurité.

Tu es développeur backend Java senior.
Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN.

> Je développe la partie backend du module M2 « Rendez-vous, Planning & Notifications ».
> Fichiers concernés :
> - entity : RendezVous, StatutRendezVous, Notification, TypeNotification, PlageHoraire (nouveau), Indisponibilite (nouveau), CanalNotification (nouveau)
> - controller : RendezVousController, NotificationController
> - service : RendezVousService, RappelRdvService, NotificationService, PlanningService (nouveau)
>
> Travail demandé :
> 1. Entités JPA : RendezVous, StatutRendezVous, Notification, TypeNotification, PlageHoraire (nouveau), Indisponibilite (nouveau), CanalNotification (nouveau). Ajoute les nouveaux attributs et associations du diagramme de classes du module.
> 2. Repositories Spring Data avec les requêtes nécessaires.
> 3. Services métier, avec @Transactional dès que plusieurs entités sont modifiées. Règles métier à respecter :
> - créneaux libres = plages horaires du médecin − rendez-vous existants − indisponibilités ;
> - interdire la double réservation : vérification dans le service, contrainte d'unicité (medecin_id, date_heure) et réponse 409 ;
> - annulation ou report par le patient autorisés jusqu'à 24 h avant le rendez-vous ;
> - rappel unique 24 h avant : ajouter rappelEnvoye à RendezVous et ne traiter que rappelEnvoye = false (RappelRdvService renvoie aujourd'hui le même rappel à chaque exécution horaire) ;
> - notifier patient et médecin à chaque changement de statut ; statut ABSENT pour les absences ;
> - NotificationService.notifier(utilisateur, type, titre, message, referenceType, referenceId) est utilisé par tous les modules : ne pas changer sa signature.
> 4. Contrôleurs REST exposant exactement ces endpoints, sécurisés avec @PreAuthorize :
> - GET /api/rendez-vous — Liste complète (ADMIN, SECRETAIRE)
> - GET /api/rendez-vous/patient/{id} · /medecin/{id} — RDV d'un patient / d'un médecin (Selon le rôle)
> - POST /api/rendez-vous — Créer un RDV (+ notifications) (PATIENT, SECRETAIRE, ADMIN)
> - PUT / DELETE /api/rendez-vous/{id} — Modifier, confirmer / supprimer (SECRETAIRE, MEDECIN, ADMIN)
> - GET /api/rendez-vous/creneaux?medecinId&date — Créneaux libres (Tous)
> - PUT /api/rendez-vous/{id}/annuler · /reporter · /absent — Annuler, reporter, marquer absent (Selon le rôle)
> - GET / POST / DELETE /api/medecins/{id}/plages — Plages horaires (MEDECIN, ADMIN)
> - GET / POST / DELETE /api/medecins/{id}/indisponibilites — Congés et absences (MEDECIN, ADMIN)
> - GET /api/notifications/utilisateur/{uid}[/non-lues | /count] — Notifications, non lues, compteur (Tous (soi-même))
> - PUT /api/notifications/{id}/lire · /utilisateur/{uid}/lire-tout — Marquer comme lue(s) (Tous (soi-même))
> - DELETE /api/notifications/{id} — Supprimer une notification (Tous (soi-même))
> 5. DTO d'entrée et de sortie (ne jamais renvoyer directement une entité) avec validation jakarta.validation.
> 6. Erreurs gérées par GlobalExceptionHandler : 400 (validation), 403, 404, 409 (conflit).
>
> Contraintes : ne modifie pas les fichiers des autres modules (demande-moi si un contrat doit évoluer), utilise NotificationService.notifier(...) pour les notifications, aucun secret dans le code, endpoints visibles dans Swagger. Donne le code complet de chaque fichier avec son chemin.

Résultat attendu : Code Spring Boot complet du module, prêt à être relu, testé puis intégré par pull request.

4.4 Prompt M2-P3 — Développement frontend

M2-P3 Développement frontend

Développement — services Angular, écrans, routes et menus.

> Tu es développeur frontend Angular 17 (composants standalone, formulaires réactifs, HttpClient, RxJS).
> Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN. L'intercepteur authInterceptor ajoute le jeton JWT ; authGuard et roleGuard protègent les routes.
>
> Je développe l'interface du module M2 « Rendez-vous, Planning & Notifications ».
> Fichiers concernés :
> - pages/front : rendez-vous-front, mes-notifications
> - pages/back : rendez-vous-back, plannings, notifications, disponibilites (nouveau)
> - pages/labo : labo-notifications ; shared : notifications-list ; badge dans les layouts
> - core : rendez-vous.service, notification.service (nouveaux)
>
> Travail demandé :
> 1. Crée core/rendez-vous.service.ts et core/notification.service.ts avec une méthode typée par endpoint du module :
> - GET /api/rendez-vous — Liste complète (ADMIN, SECRETAIRE)
> - GET /api/rendez-vous/patient/{id} · /medecin/{id} — RDV d'un patient / d'un médecin (Selon le rôle)
> - POST /api/rendez-vous — Créer un RDV (+ notifications) (PATIENT, SECRETAIRE, ADMIN)
> - PUT / DELETE /api/rendez-vous/{id} — Modifier, confirmer / supprimer (SECRETAIRE, MEDECIN, ADMIN)
> - GET /api/rendez-vous/creneaux?medecinId&date — Créneaux libres (Tous)
> - PUT /api/rendez-vous/{id}/annuler · /reporter · /absent — Annuler, reporter, marquer absent (Selon le rôle)
> - GET / POST / DELETE /api/medecins/{id}/plages — Plages horaires (MEDECIN, ADMIN)
> - GET / POST / DELETE /api/medecins/{id}/indisponibilites — Congés et absences (MEDECIN, ADMIN)
> - GET /api/notifications/utilisateur/{uid}[/non-lues | /count] — Notifications, non lues, compteur (Tous (soi-même))
> - PUT /api/notifications/{id}/lire · /utilisateur/{uid}/lire-tout — Marquer comme lue(s) (Tous (soi-même))
> - DELETE /api/notifications/{id} — Supprimer une notification (Tous (soi-même))
> 2. Crée ou mets à jour les écrans suivants :
> - prise de rendez-vous patient : médecin → date → créneaux libres → motif → confirmation ;
> - mes rendez-vous : onglets « À venir » / « Passés », annulation et report ;
> - gestion des rendez-vous (secrétaire) : création, édition, suppression, filtres par médecin, date et statut ;
> - planning : vues jour / semaine / mois avec code couleur par statut ;
> - disponibilités (nouveau) : plages horaires et indisponibilités du médecin ;
> - notifications : liste, « marquer lu », « tout lire », suppression, badge du nombre de non lues (toutes les 15 s).
> 3. Déclare les routes du module dans app.routes.ts (uniquement le bloc de ce module), protégées par roleGuard pour les rôles : PATIENT, SECRETAIRE, MEDECIN, ADMIN.
> 4. Ajoute les entrées de menu dans le ou les layouts concernés.
>
> Exigences d'interface : en français, responsive (mobile, tablette, ordinateur), indicateurs de chargement, messages d'erreur clairs (403, 404, 409), confirmation avant suppression, badges de statut colorés, champs avec libellés (accessibilité). Donne le code complet (TypeScript, HTML, CSS) de chaque composant.

Résultat attendu : Composants Angular du module, service HTTP, routes protégées et entrées de menu.

4.5 Prompt M2-P4 — Tests et démonstration

M2-P4 **Tests et démonstration**

Validation — tests, données de démo et préparation de la sprint review.

> Tu es ingénieur qualité logiciel.
> Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN. Tests backend avec JUnit 5, Mockito et MockMvc.
>
> Pour le module M2 « Rendez-vous, Planning & Notifications », écris :
> 1. Les tests unitaires des services du module (Mockito), cas nominaux et cas d'erreur, en particulier :
> - créneaux libres excluant les rendez-vous et les indisponibilités
> - double réservation refusée (409)
> - annulation à moins de 24 h refusée
> - rappel envoyé une seule fois par rendez-vous
> - notifications envoyées au patient et au médecin
> 2. Les tests d'intégration des endpoints (MockMvc) qui vérifient la sécurité par rôle : 401 sans jeton, 403 avec un rôle non autorisé, 200 / 201 avec le bon rôle.
> 3. Une méthode d'initialisation des données de démonstration du module à ajouter au DataSeeder, sans dupliquer les données existantes.
> 4. Le scénario pas à pas de la démonstration de fin de sprint : Démonstration : un médecin définit ses plages → un patient réserve un créneau libre → une seconde réservation du même créneau est refusée.
> 5. La checklist de la définition de « Terminé » appliquée à ce module (compilation, sécurité par rôle, Swagger, page accessible depuis le menu, données de démo, tests verts, revue de code, aucun secret).

Résultat attendu : Tests unitaires et d'intégration, données de démonstration, scénario de démo et checklist « Terminé ».

4.6 Fiche de suivi des prompts

À remplir par le responsable du module à chaque utilisation d'un prompt.

Date	Prompt	Résultat obtenu	Corrections apportées par l'équipe
	M2-P1		
	M2-P2		
	M2-P3		
	M2-P4		

Tableau 4.1 — Fiche de suivi — module M2

CHAPITRE 5 · SPRINT 3

Module M3 — Consultations, Ordonnances & Facturation

5.1 Présentation du module

SPRINT	Sprint 3 — 3 semaines
STATUT	Existant
RESPONSABLE	_____
OBJECTIF	Couvrir le parcours de soin de la consultation à la facture : dossier médical consolidé, ordonnances structurées reliées au catalogue de la pharmacie, facturation automatique et suivi des paiements.

User stories couvertes par les prompts

- US-3.1 : en tant que médecin, je veux enregistrer une consultation liée à un rendez-vous (diagnostic, observations, traitement) afin de tracer l'acte médical.
- US-3.2 : en tant que médecin, je veux consulter le dossier médical complet d'un patient afin de décider en connaissance de cause.
- US-3.3 : en tant que médecin, je veux rédiger une ordonnance structurée à partir du catalogue de médicaments afin de éviter les erreurs et permettre la délivrance (M6).
- US-3.4 : en tant que médecin ou patient, je veux télécharger l'ordonnance en PDF afin de l'imprimer ou la présenter.
- US-3.5 : en tant que patient, je veux consulter mon historique de consultations et d'ordonnances afin de suivre mon parcours de soins.
- US-3.6 : en tant que système, je veux générer automatiquement la facture à la clôture d'une consultation afin de ne plus oublier de facturer.
- US-3.7 : en tant que secrétaire ou administrateur, je veux créer, modifier et annuler une facture avec ses lignes afin de gérer la facturation.
- US-3.8 : en tant que secrétaire, je veux enregistrer un paiement total ou partiel et son mode afin de suivre les encaissements.
- US-3.9 : en tant que patient, je veux consulter et télécharger mes factures afin de connaître mes dépenses.
- US-3.10 : en tant qu'administrateur, je veux gérer le catalogue des actes médicaux et leurs tarifs afin de uniformiser la tarification.
- US-3.11 : en tant qu'administrateur, je veux consulter les statistiques financières afin de piloter l'activité.

5.2 Prompt M3-P1 — Analyse et conception

M3-P1 **Analyse et conception**

Conception — produit les besoins, les diagrammes UML et le sprint backlog du module.

> Tu es analyste-concepteur UML et Scrum Master.
> Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN.
>
> Je suis responsable du module M3 « Consultations, Ordonnances & Facturation », qui correspond au sprint 3 (3 semaines).
> Objectif du module : Couvrir le parcours de soin de la consultation à la facture : dossier médical consolidé, ordonnances structurées reliées au catalogue de la pharmacie, facturation automatique et suivi des paiements.
> Ce qui existe déjà dans le code :

- CRUD consultations et ordonnances (médicaments en texte libre)
- CRUD factures avec lignes, paiement, statistiques
- Dossier et historique du patient, « Mes factures »
- Composants OrdonnancesComponent et FacturationComponent présents mais non reliés au routage
Ce qu'il faut ajouter :
- Ordonnance structurée (LigneOrdonnance → Medicament)
- Export PDF des ordonnances et factures
- Facture générée automatiquement à la clôture
- Catalogue des actes médicaux (ActeMedical)
- Historique des paiements (Paiement, ModePaiement)

Produis en français, pour ce module uniquement :
1. Les besoins fonctionnels (formulés « Le système doit… », identifiants BF-M3-01, BF-M3-02…) et les besoins non fonctionnels qui le concernent (sécurité, performance, ergonomie, fiabilité).
2. Le diagramme de cas d'utilisation en PlantUML. Acteurs : Médecin, Secrétaire, Administrateur, Patient. Utilise les relations « include » et « extend » ; marque les cas nouveaux avec le stéréotype <<nouveau>>.
3. La description textuelle du cas « Enregistrer et clôturer une consultation » : acteurs, pré-conditions, post-conditions, scénario nominal numéroté, scénarios alternatifs.
4. Le diagramme de classes en PlantUML (attributs typés Java, multiplicités, énumérations). Les classes d'autres modules apparaissent avec le stéréotype <<externe>> : Patient, Medecin (M1), RendezVous (M2), Medicament (M6).
5. Le diagramme de séquence PlantUML du scénario « Enregistrer et clôturer une consultation » à travers les couches Angular → Controller → Service → Repository.
6. Le sprint backlog sous forme de tableau : user stories « En tant que…, je veux… afin de… », priorité (Haute / Moyenne / Basse), story points (1, 2, 3, 5, 8) et tâches techniques estimées en heures.

Contraintes : reste dans le périmètre du module et respecte les contrats avec les autres modules :
- M3 utilise Utilisateur, Role, Patient, Medecin ; sécurité JWT ; JournalAuditService.tracer(…) (fourni par M1)
- M3 utilise NotificationService.notifier(utilisateur, type, titre, message, referenceType, referenceId) (fourni par M2)
- M3 fournit à M6 : Ordonnance + LigneOrdonnance (medicament, posologie, dureeJours, quantite)
- M3 utilise GET /api/pharmacie/medicaments?q= et /disponibilite (fourni par M6)
- M3 fournit à M6 : FactureService.ajouterLignes(factureId, lignes)

Résultat attendu : Besoins BF/BNF, diagrammes de cas d'utilisation, de classes et de séquence (PlantUML), description textuelle et sprint backlog.

5.3 Prompt M3-P2 — Développement backend

M3-P2 **Développement backend**

Développement — entités, services, contrôleurs REST et sécurité.

Tu es développeur backend Java senior.
Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN.

Je développe la partie backend du module M3 « Consultations, Ordonnances & Facturation ».

Fichiers concernés :
- entity : Consultation, Ordonnance, LigneOrdonnance (nouveau), Facture, LigneFacture, StatutPaiement, ActeMedical (nouveau), Paiement (nouveau), ModePaiement (nouveau)
- controller : ConsultationController, OrdonnanceController, FactureController, ActeMedicalController (nouveau)
- service : ConsultationService, OrdonnanceService, FactureService, PdfService (nouveau)

Travail demandé :
1. Entités JPA : Consultation, Ordonnance, LigneOrdonnance (nouveau), Facture, LigneFacture, StatutPaiement, ActeMedical (nouveau), Paiement (nouveau), ModePaiement (nouveau). Ajoute les nouveaux attributs et associations du diagramme de classes du module.
2. Repositories Spring Data avec les requêtes nécessaires.
3. Services métier, avec @Transactional dès que plusieurs entités sont modifiées. Règles métier à respecter :
- la clôture d'une consultation passe le rendez-vous à TERMINE, génère une facture EN_ATTENTE au tarif de l'acte (ActeMedical) et notifie le patient (FACTURE_GENEREE) ;
- ordonnance structurée en lignes : LigneOrdonnance(medicament, posologie, dureeJours, quantite) remplace le champ texte medicaments (reprendre les anciennes données) ;
- paiements partiels : enregistrer chaque Paiement, mettre à jour le statut (PARTIELLEMENT_PAYE puis PAYE) et calculer le reste à payer ;
- numérotation automatique des factures FAC-AAAA-0001 ;
- export PDF des ordonnances et des factures (OpenPDF) ;
- exposer FactureService.ajouterLignes(factureId, lignes) pour le module M6.
4. Contrôleurs REST exposant exactement ces endpoints, sécurisés avec @PreAuthorize :
- GET / POST / PUT / DELETE /api/consultations[/{id}] — Gestion des consultations (MEDECIN, SECRETAIRE, ADMIN)
- GET /api/consultations/patient/{id} · /medecin/{id} — Consultations d'un patient / d'un médecin (Selon le rôle)
- PUT /api/consultations/{id}/cloturer — Clôturer + facture automatique (MEDECIN)
- GET /api/patients/{id}/dossier — Dossier médical consolidé (MEDECIN, PATIENT (soi-même))
- GET / POST / PUT / DELETE /api/ordonnances[/{id}] — Gestion des ordonnances (avec lignes) (MEDECIN)
- GET /api/ordonnances/{id}/pdf — Export PDF (MEDECIN, PATIENT)
- GET / POST / PUT / DELETE /api/factures[/{id}] — Gestion des factures (SECRETAIRE, ADMIN)
- POST /api/factures/{id}/paiements — Enregistrer un paiement (SECRETAIRE, ADMIN)
- GET /api/factures/{id}/pdf · /api/factures/stats — Export PDF · statistiques (Selon le rôle)
- GET / POST / PUT / DELETE /api/actes[/{id}] — Catalogue des actes médicaux (ADMIN)
5. DTO d'entrée et de sortie (ne jamais renvoyer directement une entité) avec validation jakarta.validation.
6. Erreurs gérées par GlobalExceptionHandler : 400 (validation), 403, 404, 409 (conflit).

Contraintes : ne modifie pas les fichiers des autres modules (demande-moi si un contrat doit évoluer), utilise NotificationService.notifier(...) pour les notifications, aucun secret dans le code, endpoints visibles dans Swagger. Donne le code complet de chaque fichier avec son chemin.

Résultat attendu : Code Spring Boot complet du module, prêt à être relu, testé puis intégré par pull request.

5.4 Prompt M3-P3 — Développement frontend

M3-P3 Développement frontend

Développement — services Angular, écrans, routes et menus.

Tu es développeur frontend Angular 17 (composants standalone, formulaires réactifs, HttpClient, RxJS).
Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone

(src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN. L'intercepteur authInterceptor ajoute le jeton JWT ; authGuard et roleGuard protègent les routes.

Je développe l'interface du module M3 « Consultations, Ordonnances & Facturation ».
Fichiers concernés :
- pages/back : consultations, ordonnances, factures, facturation, actes (nouveau)
- pages/front : dossier, historique, mes-factures
- core : consultation.service, facture.service (nouveaux)

Travail demandé :
1. Crée core/consultation.service.ts et core/facture.service.ts avec une méthode typée par endpoint du module :
- GET / POST / PUT / DELETE /api/consultations[/{id}] — Gestion des consultations (MEDECIN, SECRETAIRE, ADMIN)
- GET /api/consultations/patient/{id} · /medecin/{id} — Consultations d'un patient / d'un médecin (Selon le rôle)
- PUT /api/consultations/{id}/cloturer — Clôturer + facture automatique (MEDECIN)
- GET /api/patients/{id}/dossier — Dossier médical consolidé (MEDECIN, PATIENT (soi-même))
- GET / POST / PUT / DELETE /api/ordonnances[/{id}] — Gestion des ordonnances (avec lignes) (MEDECIN)
- GET /api/ordonnances/{id}/pdf — Export PDF (MEDECIN, PATIENT)
- GET / POST / PUT / DELETE /api/factures[/{id}] — Gestion des factures (SECRETAIRE, ADMIN)
- POST /api/factures/{id}/paiements — Enregistrer un paiement (SECRETAIRE, ADMIN)
- GET /api/factures/{id}/pdf · /api/factures/stats — Export PDF · statistiques (Selon le rôle)
- GET / POST / PUT / DELETE /api/actes[/{id}] — Catalogue des actes médicaux (ADMIN)
2. Crée ou mets à jour les écrans suivants :
- consultation : formulaire lié au rendez-vous, affichage du dossier médical, bouton « Clôturer » ;
- ordonnance : lignes dynamiques avec recherche de médicament (API M6) et indication de disponibilité ;
- factures et facturation : relier FacturationComponent et OrdonnancesComponent au routage (ils existent mais ne sont pas routés) ;
- paiement : mode, montant, référence ;
- actes (nouveau) : catalogue des actes médicaux et tarifs ;
- espace patient : historique, dossier, mes factures avec téléchargement PDF.
3. Déclare les routes du module dans app.routes.ts (uniquement le bloc de ce module), protégées par roleGuard pour les rôles : MEDECIN, SECRETAIRE, ADMIN, PATIENT.
4. Ajoute les entrées de menu dans le ou les layouts concernés.

Exigences d'interface : en français, responsive (mobile, tablette, ordinateur), indicateurs de chargement, messages d'erreur clairs (403, 404, 409), confirmation avant suppression, badges de statut colorés, champs avec libellés (accessibilité). Donne le code complet (TypeScript, HTML, CSS) de chaque composant.

Résultat attendu : Composants Angular du module, service HTTP, routes protégées et entrées de menu.

5.5 Prompt M3-P4 — Tests et démonstration

M3-P4 **Tests et démonstration**

Validation — tests, données de démo et préparation de la sprint review.

Tu es ingénieur qualité logiciel.
Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN. Tests backend avec JUnit 5, Mockito et MockMvc.

> Pour le module M3 « Consultations, Ordonnances & Facturation », écris :
> 1. Les tests unitaires des services du module (Mockito), cas nominaux et cas d'erreur, en particulier :
> - la clôture génère la facture au bon tarif
> - un paiement partiel puis total met à jour le statut
> - calcul du reste à payer
> - ordonnance enregistrée avec ses lignes
> - un patient ne peut pas lire les factures d'un autre patient
> 2. Les tests d'intégration des endpoints (MockMvc) qui vérifient la sécurité par rôle : 401 sans jeton, 403 avec un rôle non autorisé, 200 / 201 avec le bon rôle.
> 3. Une méthode d'initialisation des données de démonstration du module à ajouter au DataSeeder, sans dupliquer les données existantes.
> 4. Le scénario pas à pas de la démonstration de fin de sprint : Démonstration : consultation → ordonnance structurée → clôture → facture générée → paiement partiel puis total.
> 5. La checklist de la définition de « Terminé » appliquée à ce module (compilation, sécurité par rôle, Swagger, page accessible depuis le menu, données de démo, tests verts, revue de code, aucun secret).

Résultat attendu : Tests unitaires et d'intégration, données de démonstration, scénario de démo et checklist « Terminé ».

5.6 Fiche de suivi des prompts

À remplir par le responsable du module à chaque utilisation d'un prompt.

Date	Prompt	Résultat obtenu	Corrections apportées par l'équipe
	M3-P1		
	M3-P2		
	M3-P3		
	M3-P4		

Tableau 5.1 — Fiche de suivi — module M3

CHAPITRE 6 · SPRINT 4

Module M4 — Examens & Laboratoire

6.1 Présentation du module

SPRINT	Sprint 4 — 3 semaines
STATUT	Existant
RESPONSABLE	_____
OBJECTIF	Structurer le circuit des examens : prescription à partir d'un catalogue, traitement par le laboratoire, résultats saisis par paramètre avec détection automatique des valeurs anormales, publication et suivi dans le temps.

User stories couvertes par les prompts

- US-4.1 : en tant que médecin, je veux prescrire un examen à partir du catalogue (type, urgence, description) afin de obtenir les analyses nécessaires.
- US-4.2 : en tant que médecin, je veux annuler un examen non commencé afin de corriger une prescription.
- US-4.3 : en tant que laborantin, je veux consulter les examens à traiter filtrés par statut et urgence afin de prioriser mon travail.
- US-4.4 : en tant que laborantin, je veux prendre en charge un examen afin de signaler qu'il est en cours de traitement.
- US-4.5 : en tant que laborantin, je veux saisir les résultats paramètre par paramètre afin de produire un résultat structuré et exploitable.
- US-4.6 : en tant que système, je veux signaler automatiquement les valeurs hors norme afin de attirer l'attention du médecin.
- US-4.7 : en tant que laborantin, je veux joindre un compte rendu (PDF ou image) afin de fournir le document original.
- US-4.8 : en tant que laborantin, je veux publier le résultat afin de informer le patient et le médecin.
- US-4.9 : en tant que patient, je veux consulter et télécharger mes résultats afin de suivre ma santé.
- US-4.10 : en tant que médecin, je veux suivre l'évolution d'un paramètre pour un patient afin de apprécier l'évolution clinique.
- US-4.11 : en tant qu'administrateur, je veux gérer le catalogue des examens et leurs valeurs de référence afin de standardiser les analyses.
- US-4.12 : en tant qu'administrateur, je veux superviser l'activité du laboratoire (volumes, délais) afin de piloter le laboratoire.

6.2 Prompt M4-P1 — Analyse et conception

M4-P1 Analyse et conception

Conception — produit les besoins, les diagrammes UML et le sprint backlog du module.

> Tu es analyste-concepteur UML et Scrum Master.
> Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN.
>
> Je suis responsable du module M4 « Examens & Laboratoire », qui correspond au sprint 4 (3 semaines).
> Objectif du module : Structurer le circuit des examens : prescription à partir d'un catalogue, traitement par le laboratoire, résultats saisis par paramètre avec détection automatique des valeurs anormales, publication et suivi dans le temps.
> Ce qui existe déjà dans le code :
> - Prescription et CRUD des examens (statuts DEMANDE → RESULTAT_DISPONIBLE)

- Espace laboratoire /labo : examens à traiter, saisie des résultats, notifications
- Résultats en texte libre + fichier, consultation patient
- Supervision du laboratoire par l'administrateur
Ce qu'il faut ajouter :
- Catalogue TypeExamen avec ParametreExamen (valeurs de référence)
- Saisie par paramètre (ResultatParametre) et détection hors norme
- Examen urgent et laborantin responsable
- Téléversement sécurisé des comptes rendus
- Évolution d'un paramètre dans le temps

Produis en français, pour ce module uniquement :
1. Les besoins fonctionnels (formulés « Le système doit… », identifiants BF-M4-01, BF-M4-02…) et les besoins non fonctionnels qui le concernent (sécurité, performance, ergonomie, fiabilité).
2. Le diagramme de cas d'utilisation en PlantUML. Acteurs : Médecin, Administrateur, Laborantin, Patient. Utilise les relations « include » et « extend » ; marque les cas nouveaux avec le stéréotype <<nouveau>>.
3. La description textuelle du cas « Saisir et publier les résultats d'un examen » : acteurs, pré-conditions, post-conditions, scénario nominal numéroté, scénarios alternatifs.
4. Le diagramme de classes en PlantUML (attributs typés Java, multiplicités, énumérations). Les classes d'autres modules apparaissent avec le stéréotype <<externe>> : Patient, Medecin, Utilisateur (M1), Consultation (M3).
5. Le diagramme de séquence PlantUML du scénario « Saisir et publier les résultats d'un examen » à travers les couches Angular → Controller → Service → Repository.
6. Le sprint backlog sous forme de tableau : user stories « En tant que…, je veux… afin de… », priorité (Haute / Moyenne / Basse), story points (1, 2, 3, 5, 8) et tâches techniques estimées en heures.

Contraintes : reste dans le périmètre du module et respecte les contrats avec les autres modules :
- M4 utilise Utilisateur, Role, Patient, Medecin ; sécurité JWT ; JournalAuditService.tracer(...) (fourni par M1)
- M4 utilise NotificationService.notifier(utilisateur, type, titre, message, referenceType, referenceId) (fourni par M2)
- M4 fournit à M5 : ExamenLaboratoire + ResultatParametre (horsNorme)

Résultat attendu : Besoins BF/BNF, diagrammes de cas d'utilisation, de classes et de séquence (PlantUML), description textuelle et sprint backlog.

6.3 Prompt M4-P2 — Développement backend

M4-P2 **Développement backend**

Développement — entités, services, contrôleurs REST et sécurité.

Tu es développeur backend Java senior.
Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN.

Je développe la partie backend du module M4 « Examens & Laboratoire ».
Fichiers concernés :
- entity : ExamenLaboratoire, StatutExamen, TypeExamen (nouveau), ParametreExamen (nouveau), ResultatParametre (nouveau)
- controller : ExamenLaboratoireController, ExamenController, LaboController, TypeExamenController (nouveau)

- service : ExamenLaboratoireService, ExamenService, LaboService, FichierService (nouveau)

Travail demandé :
1. Entités JPA : ExamenLaboratoire, StatutExamen, TypeExamen (nouveau), ParametreExamen (nouveau), ResultatParametre (nouveau). Ajoute les nouveaux attributs et associations du diagramme de classes du module.
2. Repositories Spring Data avec les requêtes nécessaires.
3. Services métier, avec @Transactional dès que plusieurs entités sont modifiées. Règles métier à respecter :
- catalogue TypeExamen avec ses ParametreExamen (unité, valeurMin, valeurMax) ;
- saisie des résultats par paramètre (ResultatParametre) et calcul automatique de horsNorme ;
- transitions autorisées : DEMANDE → EN_COURS → RESULTAT_DISPONIBLE ; annulation uniquement au statut DEMANDE ;
- téléversement des comptes rendus : PDF, JPG ou PNG, 5 Mo maximum, stockage hors du dossier public, téléchargement contrôlé par rôle ;
- à la publication : notifier le patient et le médecin, et alerter le médecin si une valeur est hors norme ;
- enregistrer le laborantin qui a traité l'examen.
4. Contrôleurs REST exposant exactement ces endpoints, sécurisés avec @PreAuthorize :
- GET /api/examens[/{id}] — Liste / détail des examens (ADMIN, MEDECIN, SECRETAIRE, LABO)
- GET /api/examens/patient/{id} · /medecin/{id} — Examens d'un patient / d'un médecin (Selon le rôle)
- POST / PUT / DELETE /api/examens[/{id}] — Prescrire, modifier, supprimer (MEDECIN, ADMIN (PUT : LABO))
- POST /api/labo/resultats — Publier un résultat (par paramètre) (LABO, MEDECIN, ADMIN)
- PUT /api/examens/{id}/demarrer — Prise en charge par le laboratoire (LABO)
- POST /api/examens/{id}/fichier — Téléverser le compte rendu (LABO)
- GET /api/examens/patient/{id}/parametres/{pid}/evolution — Évolution d'un paramètre (MEDECIN, PATIENT)
- GET / POST / PUT / DELETE /api/types-examens[/{id}] — Catalogue des examens et paramètres (ADMIN (lecture : tous))
5. DTO d'entrée et de sortie (ne jamais renvoyer directement une entité) avec validation jakarta.validation.
6. Erreurs gérées par GlobalExceptionHandler : 400 (validation), 403, 404, 409 (conflit).

Contraintes : ne modifie pas les fichiers des autres modules (demande-moi si un contrat doit évoluer), utilise NotificationService.notifier(...) pour les notifications, aucun secret dans le code, endpoints visibles dans Swagger. Donne le code complet de chaque fichier avec son chemin.

Résultat attendu : Code Spring Boot complet du module, prêt à être relu, testé puis intégré par pull request.

6.4 Prompt M4-P3 — Développement frontend

M4-P3 Développement frontend

Développement — services Angular, écrans, routes et menus.

Tu es développeur frontend Angular 17 (composants standalone, formulaires réactifs, HttpClient, RxJS).
Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN. L'intercepteur authInterceptor ajoute le jeton JWT ; authGuard et roleGuard protègent les routes.

Je développe l'interface du module M4 « Examens & Laboratoire ».
Fichiers concernés :
- pages/back : examens, laboratoire, types-examens (nouveau)
- pages/labo : labo-examens, labo-resultats ; layouts : labo-layout

- pages/front : mes-examens ; core : examen.service (nouveau)

Travail demandé :
1. Crée core/examen.service.ts avec une méthode typée par endpoint du module :
- GET /api/examens[/{id}] — Liste / détail des examens (ADMIN, MEDECIN, SECRETAIRE, LABO)
- GET /api/examens/patient/{id} · /medecin/{id} — Examens d'un patient / d'un médecin (Selon le rôle)
- POST / PUT / DELETE /api/examens[/{id}] — Prescrire, modifier, supprimer (MEDECIN, ADMIN (PUT : LABO))
- POST /api/labo/resultats — Publier un résultat (par paramètre) (LABO, MEDECIN, ADMIN)
- PUT /api/examens/{id}/demarrer — Prise en charge par le laboratoire (LABO)
- POST /api/examens/{id}/fichier — Téléverser le compte rendu (LABO)
- GET /api/examens/patient/{id}/parametres/{pid}/evolution — Évolution d'un paramètre (MEDECIN, PATIENT)
- GET / POST / PUT / DELETE /api/types-examens[/{id}] — Catalogue des examens et paramètres (ADMIN (lecture : tous))
2. Crée ou mets à jour les écrans suivants :
- prescription : type d'examen du catalogue, case « urgent », description ;
- examens à traiter (laboratoire) : filtres par statut et urgence, tri ;
- saisie des résultats : formulaire généré à partir des paramètres du type, valeurs hors norme surlignées, téléversement ;
- mes examens (patient) : liste, détail, téléchargement ;
- évolution d'un paramètre : graphique (Chart.js) ;
- types d'examens (nouveau, administrateur) et supervision du laboratoire.
3. Déclare les routes du module dans app.routes.ts (uniquement le bloc de ce module), protégées par roleGuard pour les rôles : MEDECIN, LABO, ADMIN, PATIENT.
4. Ajoute les entrées de menu dans le ou les layouts concernés.

Exigences d'interface : en français, responsive (mobile, tablette, ordinateur), indicateurs de chargement, messages d'erreur clairs (403, 404, 409), confirmation avant suppression, badges de statut colorés, champs avec libellés (accessibilité). Donne le code complet (TypeScript, HTML, CSS) de chaque composant.

Résultat attendu : Composants Angular du module, service HTTP, routes protégées et entrées de menu.

6.5 Prompt M4-P4 — Tests et démonstration

M4-P4 **Tests et démonstration**

Validation — tests, données de démo et préparation de la sprint review.

Tu es ingénieur qualité logiciel.
Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN. Tests backend avec JUnit 5, Mockito et MockMvc.

Pour le module M4 « Examens & Laboratoire », écris :
1. Les tests unitaires des services du module (Mockito), cas nominaux et cas d'erreur, en particulier :
- transition de statut interdite refusée
- calcul de horsNorme aux bornes
- fichier de plus de 5 Mo refusé
- notifications envoyées à la publication
- évolution d'un paramètre triée par date

> 2. Les tests d'intégration des endpoints (MockMvc) qui vérifient la sécurité par rôle : 401 sans jeton, 403 avec un rôle non autorisé, 200 / 201 avec le bon rôle.
> 3. Une méthode d'initialisation des données de démonstration du module à ajouter au DataSeeder, sans dupliquer les données existantes.
> 4. Le scénario pas à pas de la démonstration de fin de sprint : Démonstration : prescription d'une NFS urgente → prise en charge → saisie par paramètre avec une valeur anormale → publication → notifications.
> 5. La checklist de la définition de « Terminé » appliquée à ce module (compilation, sécurité par rôle, Swagger, page accessible depuis le menu, données de démo, tests verts, revue de code, aucun secret).

Résultat attendu : Tests unitaires et d'intégration, données de démonstration, scénario de démo et checklist « Terminé ».

6.6 Fiche de suivi des prompts

À remplir par le responsable du module à chaque utilisation d'un prompt.

Date	Prompt	Résultat obtenu	Corrections apportées par l'équipe
	M4-P1		
	M4-P2		
	M4-P3		
	M4-P4		

Tableau 6.1 — Fiche de suivi — module M4

CHAPITRE 7 · SPRINT 5

Module M5 — IA & Aide à la décision

7.1 Présentation du module

SPRINT	Sprint 5 — 3 semaines
STATUT	Nouveau
RESPONSABLE	_____
OBJECTIF	Faire de l'intelligence artificielle un véritable outil d'aide : assistant conversationnel sécurisé côté serveur, pré-triage des symptômes relié à la prise de rendez-vous, résumé de dossier et interprétation de résultats pour le médecin, tableaux de bord et prévisions pour l'administration.

User stories couvertes par les prompts

- US-5.1 : en tant qu'utilisateur, je veux discuter avec l'assistant MediAssist avec des réponses adaptées à mon rôle afin de obtenir de l'aide rapidement.
- US-5.2 : en tant qu'utilisateur, je veux retrouver l'historique de mes conversations afin de reprendre un échange.
- US-5.3 : en tant que patient, je veux décrire mes symptômes et obtenir une spécialité, un niveau d'urgence et des médecins disponibles afin de prendre le bon rendez-vous.
- US-5.4 : en tant que patient, je veux être alerté et orienté vers les urgences en cas d'urgence vitale afin de ne pas retarder ma prise en charge.
- US-5.5 : en tant que médecin, je veux générer un résumé IA du dossier d'un patient afin de préparer la consultation rapidement.
- US-5.6 : en tant que médecin, je veux obtenir une aide à l'interprétation des résultats hors norme afin de gagner du temps d'analyse.
- US-5.7 : en tant qu'administrateur, médecin ou secrétaire, je veux consulter un tableau de bord adapté à mon rôle afin de suivre les indicateurs clés.
- US-5.8 : en tant qu'administrateur, je veux consulter les statistiques détaillées afin de analyser l'activité.
- US-5.9 : en tant qu'administrateur, je veux consulter les prévisions d'affluence et le risque d'absence afin de anticiper les ressources.
- US-5.10 : en tant qu'administrateur, je veux exporter les statistiques en PDF ou Excel afin de les partager.
- US-5.11 : en tant qu'administrateur, je veux que chaque appel IA soit journalisé et accompagné d'un avertissement afin de garantir un usage responsable.

7.2 Prompt M5-P1 — Analyse et conception

M5-P1 **Analyse et conception**

Conception — produit les besoins, les diagrammes UML et le sprint backlog du module.

> Tu es analyste-concepteur UML et Scrum Master.
> Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN.
>
> Je suis responsable du module M5 « IA & Aide à la décision », qui correspond au sprint 5 (3 semaines).
> Objectif du module : Faire de l'intelligence artificielle un véritable outil d'aide : assistant conversationnel sécurisé côté serveur, pré-triage des symptômes relié à la prise de rendez-vous, résumé de dossier et interprétation de résultats pour le médecin, tableaux de bord et prévisions pour l'administration.
> Ce qui existe déjà dans le code :

- Chatbot MediAssist (ChatbotService) avec prompt adapté au rôle
- Appel hybride : direct depuis Angular avec repli sur /api/chatbot
- Tableau de bord, statistiques et statistiques détaillées
Ce qu'il faut ajouter :
- Tous les appels IA passent par le serveur (la clé n'est plus côté navigateur)
- Historique des conversations
- Pré-triage des symptômes → spécialité, urgence, médecins
- Résumé IA du dossier, aide à l'interprétation des résultats
- Prévisions d'affluence et risque d'absence, exports

Produis en français, pour ce module uniquement :
1. Les besoins fonctionnels (formulés « Le système doit… », identifiants BF-M5-01, BF-M5-02…) et les besoins non fonctionnels qui le concernent (sécurité, performance, ergonomie, fiabilité).
2. Le diagramme de cas d'utilisation en PlantUML. Acteurs : Utilisateur authentifié, Patient, Médecin, Administrateur, API IA Groq (acteur secondaire). Utilise les relations « include » et « extend » ; marque les cas nouveaux avec le stéréotype <<nouveau>>.
3. La description textuelle du cas « Pré-triage des symptômes » : acteurs, pré-conditions, post-conditions, scénario nominal numéroté, scénarios alternatifs.
4. Le diagramme de classes en PlantUML (attributs typés Java, multiplicités, énumérations). Les classes d'autres modules apparaissent avec le stéréotype <<externe>> : Utilisateur, Patient, Medecin (M1), ExamenLaboratoire (M4).
5. Le diagramme de séquence PlantUML du scénario « Pré-triage des symptômes » à travers les couches Angular → Controller → Service → Repository.
6. Le sprint backlog sous forme de tableau : user stories « En tant que…, je veux… afin de… », priorité (Haute / Moyenne / Basse), story points (1, 2, 3, 5, 8) et tâches techniques estimées en heures.

Contraintes : reste dans le périmètre du module et respecte les contrats avec les autres modules :
- M5 utilise Utilisateur, Role, Patient, Medecin ; sécurité JWT ; JournalAuditService.tracer(…) (fourni par M1)
- M5 utilise NotificationService.notifier(utilisateur, type, titre, message, referenceType, referenceId) (fourni par M2)
- M5 utilise Historique des RDV et statut ABSENT (fourni par M2)
- M5 utilise ExamenLaboratoire + ResultatParametre (horsNorme) (fourni par M4)
- M5 fournit à M2 : Médecin suggéré par le triage

Résultat attendu : Besoins BF/BNF, diagrammes de cas d'utilisation, de classes et de séquence (PlantUML), description textuelle et sprint backlog.

7.3 Prompt M5-P2 — Développement backend

M5-P2 Développement backend

Développement — entités, services, contrôleurs REST et sécurité.

Tu es développeur backend Java senior.
Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN.

Je développe la partie backend du module M5 « IA & Aide à la décision ».

Fichiers concernés :
- entity : ConversationIA, MessageIA, TriageSymptomes, AnalyseIA + énumérations (nouveaux)
- controller : ChatbotController, StatistiqueController, AssistantIAController (nouveau)
- service : StatistiqueService, AssistantIAService (remplace ChatbotService), GroqClient, PrevisionService (nouveaux)

Travail demandé :
1. Entités JPA : ConversationIA, MessageIA, TriageSymptomes, AnalyseIA + énumérations (nouveaux). Ajoute les nouveaux attributs et associations du diagramme de classes du module.
2. Repositories Spring Data avec les requêtes nécessaires.
3. Services métier, avec @Transactional dès que plusieurs entités sont modifiées. Règles métier à respecter :
- tous les appels à l'API Groq passent par le serveur (GroqClient) ; la clé est lue dans une variable d'environnement et n'apparaît jamais dans le code Angular ;
- délai maximal de 10 s et message de repli en cas d'erreur : l'IA ne bloque jamais un processus métier ;
- pseudonymisation : n'envoyer que l'âge, le sexe et les antécédents, jamais le nom ni le numéro de dossier ;
- demander une réponse JSON structurée (spécialité, niveau d'urgence, justification) et la valider avant usage ;
- si le niveau est URGENCE_VITALE : afficher une alerte d'orientation vers les urgences et ne proposer aucun rendez-vous ;
- journaliser chaque appel (modèle, utilisateur, durée) ; les analyses (AnalyseIA) doivent être validées par le médecin ;
- prévisions calculées à partir de l'historique des rendez-vous (moyennes mobiles, taux d'absence par créneau à partir du statut ABSENT).
4. Contrôleurs REST exposant exactement ces endpoints, sécurisés avec @PreAuthorize :
- POST /api/chatbot — Message à l'assistant (côté serveur uniquement) (Tous)
- GET /api/ia/conversations[/{id}/messages] — Historique des conversations (Tous (soi-même))
- POST /api/ia/triage — Pré-triage des symptômes (PATIENT)
- POST /api/ia/resume-dossier/{patientId} — Résumé IA du dossier (MEDECIN)
- POST /api/ia/interpretation/{examenId} — Aide à l'interprétation (MEDECIN)
- GET /api/statistiques · /api/statistiques-detail — KPI et statistiques détaillées (ADMIN, MEDECIN, SECRETAIRE)
- GET /api/ia/previsions?semaine= — Prévisions d'affluence et d'absences (ADMIN)
- GET /api/statistiques/export?format=pdf|xlsx — Export des statistiques (ADMIN)
5. DTO d'entrée et de sortie (ne jamais renvoyer directement une entité) avec validation jakarta.validation.
6. Erreurs gérées par GlobalExceptionHandler : 400 (validation), 403, 404, 409 (conflit).

Contraintes : ne modifie pas les fichiers des autres modules (demande-moi si un contrat doit évoluer), utilise NotificationService.notifier(...) pour les notifications, aucun secret dans le code, endpoints visibles dans Swagger. Donne le code complet de chaque fichier avec son chemin.

Résultat attendu : Code Spring Boot complet du module, prêt à être relu, testé puis intégré par pull request.

7.4 Prompt M5-P3 — Développement frontend

M5-P3 Développement frontend

Développement — services Angular, écrans, routes et menus.

Tu es développeur frontend Angular 17 (composants standalone, formulaires réactifs, HttpClient, RxJS).
Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO,

PHARMACIEN. L'intercepteur authInterceptor ajoute le jeton JWT ; authGuard et roleGuard protègent les routes.

Je développe l'interface du module M5 « IA & Aide à la décision ».
Fichiers concernés :
- shared : chatbot (appel direct à Groq supprimé), medecin-widget
- pages/back : dashboard, statistiques, statistiques-detail, previsions (nouveau)
- pages/front : triage (nouveau) ; panneau « Résumé IA » dans consultations ; core : ia.service (nouveau)

Travail demandé :
1. Crée core/ia.service.ts avec une méthode typée par endpoint du module :
- POST /api/chatbot — Message à l'assistant (côté serveur uniquement) (Tous)
- GET /api/ia/conversations[/{id}/messages] — Historique des conversations (Tous (soi-même))
- POST /api/ia/triage — Pré-triage des symptômes (PATIENT)
- POST /api/ia/resume-dossier/{patientId} — Résumé IA du dossier (MEDECIN)
- POST /api/ia/interpretation/{examenId} — Aide à l'interprétation (MEDECIN)
- GET /api/statistiques · /api/statistiques-detail — KPI et statistiques détaillées (ADMIN, MEDECIN, SECRETAIRE)
- GET /api/ia/previsions?semaine= — Prévisions d'affluence et d'absences (ADMIN)
- GET /api/statistiques/export?format=pdf|xlsx — Export des statistiques (ADMIN)
2. Crée ou mets à jour les écrans suivants :
- chatbot : supprimer l'appel direct à Groq, passer par /api/chatbot, panneau d'historique des conversations ;
- pré-triage (nouveau, patient) : saisie des symptômes → spécialité, urgence, médecins suggérés, bouton « Prendre RDV » qui préremplit l'écran du module M2 ;
- panneau « Résumé IA » dans l'écran de consultation et aide à l'interprétation dans le détail d'un examen, avec bouton « Valider » ;
- tableau de bord adapté au rôle, statistiques détaillées, prévisions (graphiques) et export PDF / Excel ;
- bandeau « Ces suggestions ne remplacent pas un avis médical » sur toutes les réponses IA.
3. Déclare les routes du module dans app.routes.ts (uniquement le bloc de ce module), protégées par roleGuard pour les rôles : tous les rôles (chatbot), PATIENT (triage), MEDECIN (résumé, interprétation), ADMIN (statistiques, prévisions).
4. Ajoute les entrées de menu dans le ou les layouts concernés.

Exigences d'interface : en français, responsive (mobile, tablette, ordinateur), indicateurs de chargement, messages d'erreur clairs (403, 404, 409), confirmation avant suppression, badges de statut colorés, champs avec libellés (accessibilité). Donne le code complet (TypeScript, HTML, CSS) de chaque composant.

Résultat attendu : Composants Angular du module, service HTTP, routes protégées et entrées de menu.

7.5 Prompt M5-P4 — Tests et démonstration

M5-P4 **Tests et démonstration**

Validation — tests, données de démo et préparation de la sprint review.

Tu es ingénieur qualité logiciel.
Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN. Tests backend avec JUnit 5, Mockito et MockMvc.

Pour le module M5 « IA & Aide à la décision », écris :

> 1. Les tests unitaires des services du module (Mockito), cas nominaux et cas d'erreur, en particulier :
> - client Groq simulé : réponse valide
> - réponse JSON invalide → repli
> - délai dépassé → repli
> - URGENCE_VITALE → aucun rendez-vous proposé
> - aucun nom ni n° de dossier dans le prompt envoyé
> - chaque appel est journalisé
> 2. Les tests d'intégration des endpoints (MockMvc) qui vérifient la sécurité par rôle : 401 sans jeton, 403 avec un rôle non autorisé, 200 / 201 avec le bon rôle.
> 3. Une méthode d'initialisation des données de démonstration du module à ajouter au DataSeeder, sans dupliquer les données existantes.
> 4. Le scénario pas à pas de la démonstration de fin de sprint : Démonstration : symptômes → spécialité et urgence → médecin suggéré → réservation (M2).
> 5. La checklist de la définition de « Terminé » appliquée à ce module (compilation, sécurité par rôle, Swagger, page accessible depuis le menu, données de démo, tests verts, revue de code, aucun secret).

Résultat attendu : Tests unitaires et d'intégration, données de démonstration, scénario de démo et checklist « Terminé ».

7.6 Fiche de suivi des prompts

À remplir par le responsable du module à chaque utilisation d'un prompt.

Date	Prompt	Résultat obtenu	Corrections apportées par l'équipe
	M5-P1		
	M5-P2		
	M5-P3		
	M5-P4		

Tableau 7.1 — Fiche de suivi — module M5

CHAPITRE 8 · SPRINT 6

Module M6 — Pharmacie & Stock

8.1 Présentation du module

SPRINT	Sprint 6 — 3 semaines
STATUT	Nouveau
RESPONSABLE	_____
OBJECTIF	Créer l'espace pharmacie : catalogue de médicaments, gestion des lots et des stocks, délivrance des ordonnances structurées selon la règle FEFO, facturation des médicaments et alertes automatiques.

User stories couvertes par les prompts

- US-6.1 : en tant qu'administrateur, je veux créer des comptes pharmacien et leur ouvrir l'espace /pharmacie afin de donner accès au module.
- US-6.2 : en tant que pharmacien, je veux gérer le catalogue des médicaments (DCI, forme, dosage, prix, seuil) afin de référencer les produits.
- US-6.3 : en tant que pharmacien, je veux gérer les fournisseurs afin de savoir qui livre quoi.
- US-6.4 : en tant que pharmacien, je veux enregistrer une réception par lot (quantité, péremption, fournisseur) afin de alimenter le stock.
- US-6.5 : en tant que pharmacien ou administrateur, je veux consulter l'état du stock en temps réel afin de connaître les quantités disponibles.
- US-6.6 : en tant que pharmacien, je veux consulter les ordonnances à délivrer afin de traiter les demandes.
- US-6.7 : en tant que pharmacien, je veux délivrer une ordonnance, totalement ou partiellement, avec prélèvement FEFO afin de servir le patient en écoulant d'abord les lots les plus proches de la péremption.
- US-6.8 : en tant que système, je veux ajouter les médicaments délivrés à la facture du patient afin de facturer la délivrance.
- US-6.9 : en tant que système, je veux alerter quand un médicament passe sous son seuil ou qu'un lot expire sous 30 jours afin de éviter ruptures et pertes.
- US-6.10 : en tant que pharmacien, je veux ajuster le stock après inventaire et consulter l'historique des mouvements afin de garantir la traçabilité.
- US-6.11 : en tant que médecin, je veux voir la disponibilité d'un médicament lors de la prescription afin de prescrire un produit disponible.
- US-6.12 : en tant que patient, je veux suivre l'état de délivrance de mon ordonnance afin de savoir quand récupérer mes médicaments.

8.2 Prompt M6-P1 — Analyse et conception

M6-P1 **Analyse et conception**

Conception — produit les besoins, les diagrammes UML et le sprint backlog du module.

> Tu es analyste-concepteur UML et Scrum Master.
> Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN.
>
> Je suis responsable du module M6 « Pharmacie & Stock », qui correspond au sprint 6 (3 semaines).
> Objectif du module : Créer l'espace pharmacie : catalogue de médicaments, gestion des lots et des stocks,

délivrance des ordonnances structurées selon la règle FEFO, facturation des médicaments et alertes automatiques.
Ce qui existe déjà dans le code :
- Aucun : module entièrement nouveau
- Point d'appui : ordonnances (M3), notifications (M2), factures (M3), modèle de l'espace /labo pour le layout
Ce qu'il faut ajouter :
- Rôle PHARMACIEN et espace /pharmacie
- Catalogue Medicament, Fournisseur, LotMedicament
- Mouvements de stock (entrée, sortie, ajustement, péremption)
- Délivrance totale / partielle (FEFO)
- Alertes stock bas et péremption (tâche planifiée)

Produis en français, pour ce module uniquement :
1. Les besoins fonctionnels (formulés « Le système doit… », identifiants BF-M6-01, BF-M6-02…) et les besoins non fonctionnels qui le concernent (sécurité, performance, ergonomie, fiabilité).
2. Le diagramme de cas d'utilisation en PlantUML. Acteurs : Pharmacien, Administrateur, Médecin, Patient, Système (planificateur). Utilise les relations « include » et « extend » ; marque les cas nouveaux avec le stéréotype <<nouveau>>.
3. La description textuelle du cas « Délivrer une ordonnance » : acteurs, pré-conditions, post-conditions, scénario nominal numéroté, scénarios alternatifs.
4. Le diagramme de classes en PlantUML (attributs typés Java, multiplicités, énumérations). Les classes d'autres modules apparaissent avec le stéréotype <<externe>> : Ordonnance, LigneOrdonnance, Facture (M3), Utilisateur (M1).
5. Le diagramme de séquence PlantUML du scénario « Délivrer une ordonnance » à travers les couches Angular → Controller → Service → Repository.
6. Le sprint backlog sous forme de tableau : user stories « En tant que…, je veux… afin de… », priorité (Haute / Moyenne / Basse), story points (1, 2, 3, 5, 8) et tâches techniques estimées en heures.

Contraintes : reste dans le périmètre du module et respecte les contrats avec les autres modules :
- M6 utilise Utilisateur, Role, Patient, Medecin ; sécurité JWT ; JournalAuditService.tracer(…) (fourni par M1)
- M6 utilise NotificationService.notifier(utilisateur, type, titre, message, referenceType, referenceId) (fourni par M2)
- M6 utilise Ordonnance + LigneOrdonnance (medicament, posologie, dureeJours, quantite) (fourni par M3)
- M6 fournit à M3 : GET /api/pharmacie/medicaments?q= et /disponibilite
- M6 utilise FactureService.ajouterLignes(factureId, lignes) (fourni par M3)

Résultat attendu : Besoins BF/BNF, diagrammes de cas d'utilisation, de classes et de séquence (PlantUML), description textuelle et sprint backlog.

8.3 Prompt M6-P2 — Développement backend

M6-P2 **Développement backend**

Développement — entités, services, contrôleurs REST et sécurité.

Tu es développeur backend Java senior.
Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN.

> Je développe la partie backend du module M6 « Pharmacie & Stock ».
> Fichiers concernés :
> - entity : Medicament, LotMedicament, MouvementStock, Fournisseur, Delivrance, LigneDelivrance, TypeMouvement, StatutDelivrance (tous nouveaux)
> - controller : PharmacieController (nouveau)
> - service : MedicamentService, StockService, DelivranceService, AlerteStockService (@Scheduled) (nouveaux)
>
> Travail demandé :
> 1. Entités JPA : Medicament, LotMedicament, MouvementStock, Fournisseur, Delivrance, LigneDelivrance, TypeMouvement, StatutDelivrance (tous nouveaux). Ajoute les nouveaux attributs et associations du diagramme de classes du module.
> 2. Repositories Spring Data avec les requêtes nécessaires.
> 3. Services métier, avec @Transactional dès que plusieurs entités sont modifiées. Règles métier à respecter :
> - le stock d'un médicament est la somme des quantiteRestante de ses lots non périmés (pas de champ dupliqué) ;
> - délivrance selon la règle FEFO (premier périmé, premier sorti), dans une transaction unique : tout ou rien ;
> - délivrance PARTIELLE si le stock est insuffisant, le reste à servir est conservé ;
> - chaque opération (réception, délivrance, ajustement, péremption) crée un MouvementStock ;
> - tâche @Scheduled quotidienne : alerte ALERTE_STOCK sous le seuil et pour les lots périmant dans moins de 30 jours ;
> - facturation des médicaments délivrés via FactureService.ajouterLignes (module M3) ;
> - espace /pharmacie réservé aux rôles PHARMACIEN et ADMIN.
> 4. Contrôleurs REST exposant exactement ces endpoints, sécurisés avec @PreAuthorize :
> - GET / POST / PUT / DELETE /api/pharmacie/medicaments[/{id}] — Catalogue (recherche ?q= utilisée par M3) (PHARMACIEN (lecture : MEDECIN))
> - GET /api/pharmacie/medicaments/{id}/disponibilite — Disponibilité d'un médicament (MEDECIN, PHARMACIEN)
> - GET / POST / PUT / DELETE /api/pharmacie/fournisseurs[/{id}] — Fournisseurs (PHARMACIEN)
> - POST /api/pharmacie/receptions — Réception d'un lot (PHARMACIEN)
> - GET /api/pharmacie/stock — État du stock (PHARMACIEN, ADMIN)
> - POST /api/pharmacie/ajustements — Ajustement d'inventaire (PHARMACIEN)
> - GET /api/pharmacie/mouvements — Historique des mouvements (PHARMACIEN, ADMIN)
> - GET /api/pharmacie/ordonnances?statut=A_DELIVRER — Ordonnances à délivrer (PHARMACIEN)
> - POST /api/pharmacie/delivrances — Délivrer une ordonnance (PHARMACIEN)
> - GET /api/pharmacie/alertes — Alertes stock et péremption (PHARMACIEN, ADMIN)
> 5. DTO d'entrée et de sortie (ne jamais renvoyer directement une entité) avec validation jakarta.validation.
> 6. Erreurs gérées par GlobalExceptionHandler : 400 (validation), 403, 404, 409 (conflit).
>
> Contraintes : ne modifie pas les fichiers des autres modules (demande-moi si un contrat doit évoluer), utilise NotificationService.notifier(...) pour les notifications, aucun secret dans le code, endpoints visibles dans Swagger. Donne le code complet de chaque fichier avec son chemin.

Résultat attendu : Code Spring Boot complet du module, prêt à être relu, testé puis intégré par pull request.

8.4 Prompt M6-P3 — Développement frontend

M6-P3 Développement frontend

Développement — services Angular, écrans, routes et menus.

> Tu es développeur frontend Angular 17 (composants standalone, formulaires réactifs, HttpClient, RxJS).
> Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA,

MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone (src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN. L'intercepteur authInterceptor ajoute le jeton JWT ; authGuard et roleGuard protègent les routes.

Je développe l'interface du module M6 « Pharmacie & Stock ».
Fichiers concernés :
- layouts : pharmacie-layout (nouveau, sur le modèle de labo-layout)
- pages/pharmacie : medicaments, stock, receptions, ordonnances, delivrance, fournisseurs, mouvements, alertes (nouveaux)
- core : pharmacie.service (nouveau)

Travail demandé :
1. Crée core/pharmacie.service.ts avec une méthode typée par endpoint du module :
- GET / POST / PUT / DELETE /api/pharmacie/medicaments[/{id}] — Catalogue (recherche ?q= utilisée par M3) (PHARMACIEN (lecture : MEDECIN))
- GET /api/pharmacie/medicaments/{id}/disponibilite — Disponibilité d'un médicament (MEDECIN, PHARMACIEN)
- GET / POST / PUT / DELETE /api/pharmacie/fournisseurs[/{id}] — Fournisseurs (PHARMACIEN)
- POST /api/pharmacie/receptions — Réception d'un lot (PHARMACIEN)
- GET /api/pharmacie/stock — État du stock (PHARMACIEN, ADMIN)
- POST /api/pharmacie/ajustements — Ajustement d'inventaire (PHARMACIEN)
- GET /api/pharmacie/mouvements — Historique des mouvements (PHARMACIEN, ADMIN)
- GET /api/pharmacie/ordonnances?statut=A_DELIVRER — Ordonnances à délivrer (PHARMACIEN)
- POST /api/pharmacie/delivrances — Délivrer une ordonnance (PHARMACIEN)
- GET /api/pharmacie/alertes — Alertes stock et péremption (PHARMACIEN, ADMIN)
2. Crée ou mets à jour les écrans suivants :
- pharmacie-layout (nouveau, sur le modèle de labo-layout) avec menu et badge d'alertes ;
- médicaments : catalogue avec recherche ; fournisseurs ;
- stock : quantités par médicament et par lot, indicateurs « sous le seuil » et « péremption proche » ;
- réceptions : saisie d'un lot (quantité, date de péremption, fournisseur) ;
- ordonnances à délivrer et écran de délivrance (quantités, lots prélevés, bon imprimable) ;
- mouvements et alertes ; statut de délivrance visible dans l'historique du patient.
3. Déclare les routes du module dans app.routes.ts (uniquement le bloc de ce module), protégées par roleGuard pour les rôles : PHARMACIEN, ADMIN (et MEDECIN en lecture pour la disponibilité).
4. Ajoute les entrées de menu dans le ou les layouts concernés.

Exigences d'interface : en français, responsive (mobile, tablette, ordinateur), indicateurs de chargement, messages d'erreur clairs (403, 404, 409), confirmation avant suppression, badges de statut colorés, champs avec libellés (accessibilité). Donne le code complet (TypeScript, HTML, CSS) de chaque composant.

Résultat attendu : Composants Angular du module, service HTTP, routes protégées et entrées de menu.

8.5 Prompt M6-P4 — Tests et démonstration

M6-P4 **Tests et démonstration**

Validation — tests, données de démo et préparation de la sprint review.

Tu es ingénieur qualité logiciel.
Contexte : Projet MediCare : plateforme web de gestion d'une clinique médicale. Backend Spring Boot 3.2.5 / Java 17 (package com.clinique.gestion : entity, repository, service, controller, dto, security, config), Spring Data JPA, MySQL 8, Spring Security avec JWT et @PreAuthorize. Frontend Angular 17 en composants standalone

(src/app/pages, src/app/layouts, src/app/core). Rôles : ADMIN, MEDECIN, SECRETAIRE, PATIENT, LABO, PHARMACIEN. Tests backend avec JUnit 5, Mockito et MockMvc.

Pour le module M6 « Pharmacie & Stock », écris :
1. Les tests unitaires des services du module (Mockito), cas nominaux et cas d'erreur, en particulier :
- FEFO prélève d'abord le lot le plus proche de la péremption
- les lots périmés sont ignorés
- délivrance partielle si stock insuffisant
- annulation complète (rollback) en cas d'erreur
- alerte générée sous le seuil
- un MouvementStock est créé à chaque opération
2. Les tests d'intégration des endpoints (MockMvc) qui vérifient la sécurité par rôle : 401 sans jeton, 403 avec un rôle non autorisé, 200 / 201 avec le bon rôle.
3. Une méthode d'initialisation des données de démonstration du module à ajouter au DataSeeder, sans dupliquer les données existantes.
4. Le scénario pas à pas de la démonstration de fin de sprint : Démonstration : réception de deux lots → ordonnance de M3 → délivrance FEFO (le lot le plus proche de la péremption sort en premier) → facture complétée.
5. La checklist de la définition de « Terminé » appliquée à ce module (compilation, sécurité par rôle, Swagger, page accessible depuis le menu, données de démo, tests verts, revue de code, aucun secret).

Résultat attendu : Tests unitaires et d'intégration, données de démonstration, scénario de démo et checklist « Terminé ».

8.6 Fiche de suivi des prompts

À remplir par le responsable du module à chaque utilisation d'un prompt.

Date	Prompt	Résultat obtenu	Corrections apportées par l'équipe
	M6-P1		
	M6-P2		
	M6-P3		
	M6-P4		

Tableau 8.1 — Fiche de suivi — module M6